

DevOps Engineer Technical Assignment Report

Pavan Kumar Goli

AWS Free Tier Production-like Deployment

1. Git Repository

The project source code was maintained using Git and hosted on GitHub to enable version control and continuous integration. GitHub served as the central repository for storing the application source code, configuration files, deployment scripts, and GitHub Actions workflow. Version control allowed changes to be tracked efficiently while providing a reliable backup of the project.

The repository contains the Node.js application developed using Express.js, the package configuration files, and the GitHub Actions workflow responsible for automating deployments. Every code modification was committed with meaningful commit messages and pushed to the main branch. This approach ensured proper project management and simplified collaboration.

The repository structure includes the application source code, dependency files, workflow configuration, and documentation. GitHub also provides a history of commits, allowing previous versions of the application to be restored whenever necessary.

Repository Link:

<https://github.com/GoliPavan888/devops-technical-assignment>

2. Deployment Guide

The deployment process was performed using AWS Free Tier services. Initially, an Ubuntu 24.04 LTS EC2 instance was launched in the AWS Management Console. Appropriate Security Groups were configured to allow inbound traffic on ports 22 (SSH), 80 (HTTP), 443 (HTTPS), and 3000 for application testing.

The EC2 instance was accessed securely using MobaXterm with an SSH key pair. After connecting, the operating system packages were updated, and Node.js, Git, Nginx, and PM2 were installed.

The application source code was cloned from GitHub. Required Node.js dependencies were installed using npm, and the application was started with PM2 to ensure continuous availability. Nginx was configured as a reverse proxy to forward incoming HTTP requests to the Node.js application running on port 3000.

Finally, the application was verified by accessing the EC2 public IP address through a web browser.

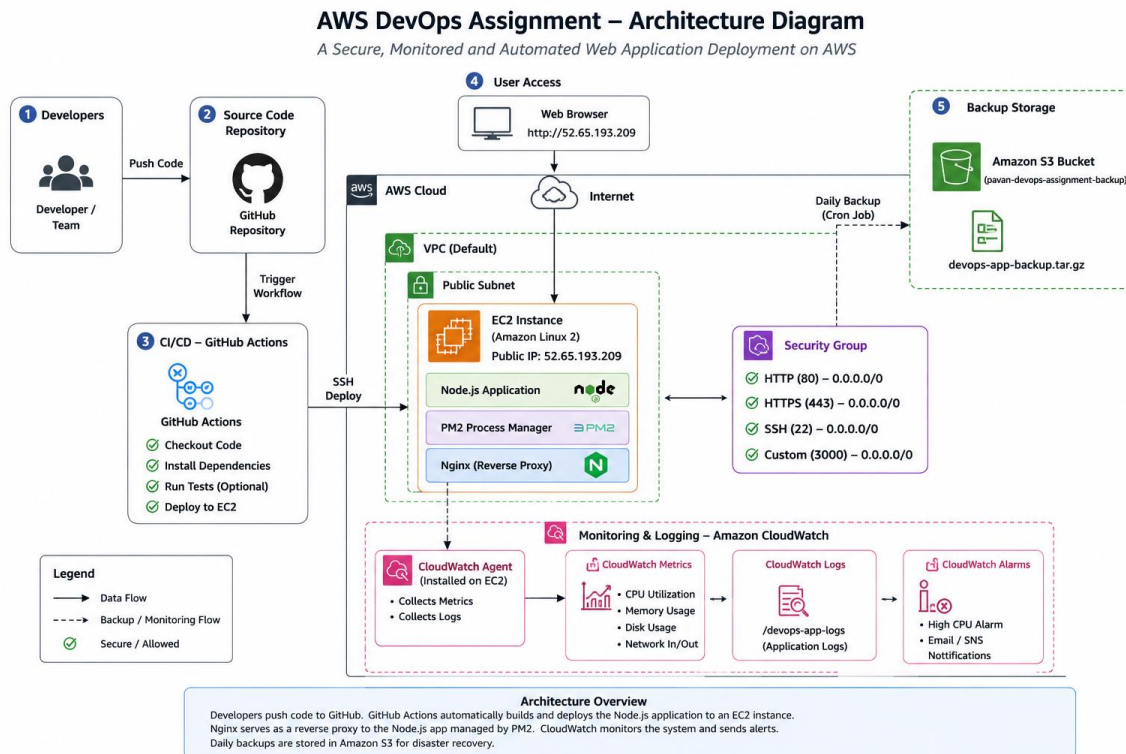
3. Architecture Diagram

System Architecture

The system architecture follows a simple production-like deployment model using AWS cloud services. Developers push source code to a GitHub repository. Every push automatically triggers a GitHub Actions workflow, which establishes a secure SSH connection with the EC2 instance and deploys the updated application.

The application is hosted on an Ubuntu EC2 instance running Node.js with Express.js. PM2 manages the Node.js process to ensure continuous execution. Nginx functions as a reverse proxy, forwarding HTTP requests from users to the application.

Amazon S3 is used to store application backups, while Amazon CloudWatch collects system metrics, application logs, and generates alarms for monitoring resource utilization.



This architecture provides automation, monitoring, backup, and secure deployment while remaining completely within AWS Free Tier limits.

4. Pipeline Configuration

CI/CD Pipeline

Continuous Integration and Continuous Deployment (CI/CD) were implemented using GitHub Actions. A workflow configuration file was created inside the `.github/workflows` directory.

Whenever changes are pushed to the main branch, GitHub Actions automatically starts a deployment workflow. The workflow checks out the latest source code, establishes a secure SSH connection to the EC2 instance, pulls the latest changes from GitHub, installs required dependencies using npm, and restarts the Node.js application using PM2.

This automation eliminates manual deployment tasks, reduces human error, and ensures that the latest version of the application is always available on the server.

GitHub Actions also provides execution logs, making it easier to monitor deployment success or troubleshoot failures.

5. Monitoring Screenshots

Monitoring and Logging

Amazon CloudWatch was configured to continuously monitor the EC2 instance and application performance. The CloudWatch Agent was installed on the EC2 instance to collect CPU utilization, memory usage, disk utilization, and network traffic metrics.

A CloudWatch Dashboard was created to display all important metrics in a single location. This dashboard enables administrators to quickly identify abnormal resource usage and monitor the overall health of the application.

A HighCPUAlarm was configured to notify administrators whenever CPU utilization exceeds 70%. Although SNS notifications were optional, the alarm demonstrates proactive monitoring capabilities.

Application logs were also configured for centralized monitoring, allowing easier debugging and troubleshooting.

6. Load Testing Report

Load Testing Report

Application performance was evaluated using the k6 load testing tool. The objective was to determine whether the application could handle multiple concurrent users while maintaining low response times and zero failures.

The test was executed with twenty virtual users running continuously for one minute. During the test, the application processed 1200 HTTP requests successfully without generating any errors.

Test Results

- Virtual Users: 20
- Duration: 1 Minute
- Total Requests: 1200
- Throughput: 19.95 Requests per Second
- Average Response Time: 1.73 ms
- Maximum Response Time: 16.45 ms
- Error Rate: 0%

Observations

The application demonstrated excellent performance under moderate load conditions. Response times remained consistently low, and the application achieved a 100% success rate without request failures. The results indicate that the deployment is stable for small-scale workloads.

7. Security Summary

Security Implementation

Security was considered throughout the deployment process. The EC2 instance uses Security Groups as a virtual firewall to restrict network access. Only essential ports such as SSH, HTTP, HTTPS, and the application testing port were allowed.

An IAM Role was attached to the EC2 instance following the principle of least privilege. This role grants only the permissions required for Amazon S3 backup operations and CloudWatch monitoring. No AWS Access Keys were stored on the EC2 instance, reducing security risks.

The application is managed by PM2, ensuring continuous operation and automatic restart in case of failures. Nginx acts as a reverse proxy, hiding the application server from direct exposure.

Application backups are securely stored in a private Amazon S3 bucket, providing disaster recovery capability.

These security practices align with AWS recommended best practices for production environments.

Implementation Screenshots

EC2 Instance

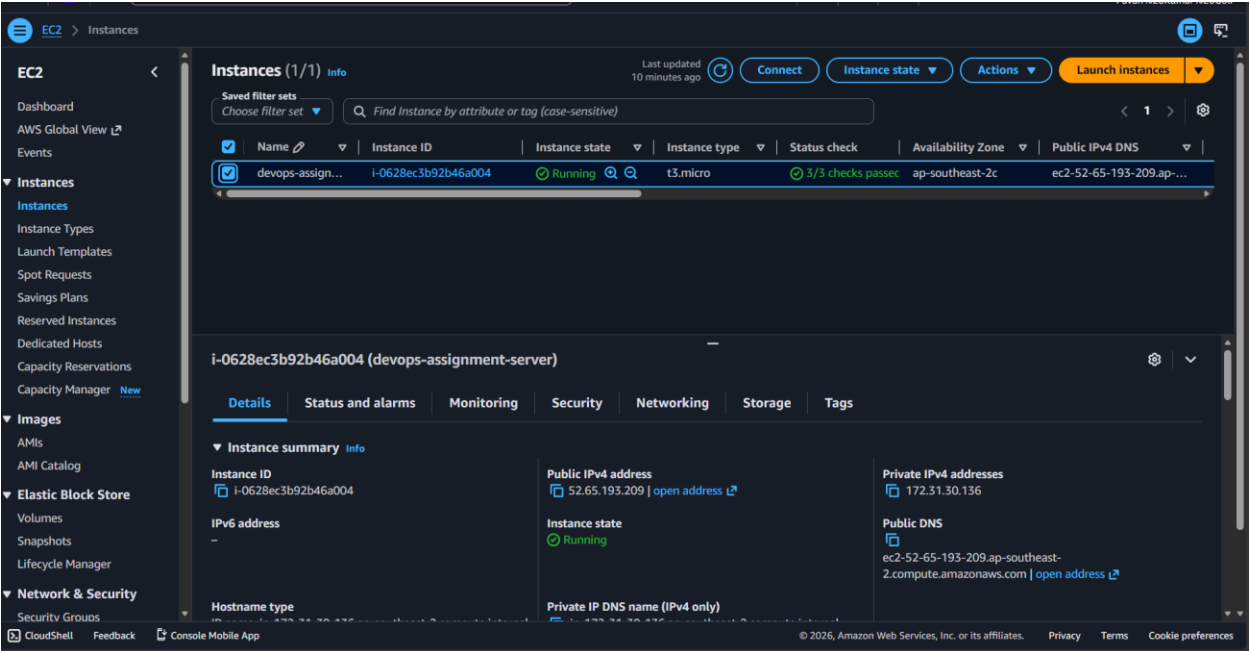


Figure: EC2 Instance

Security Group

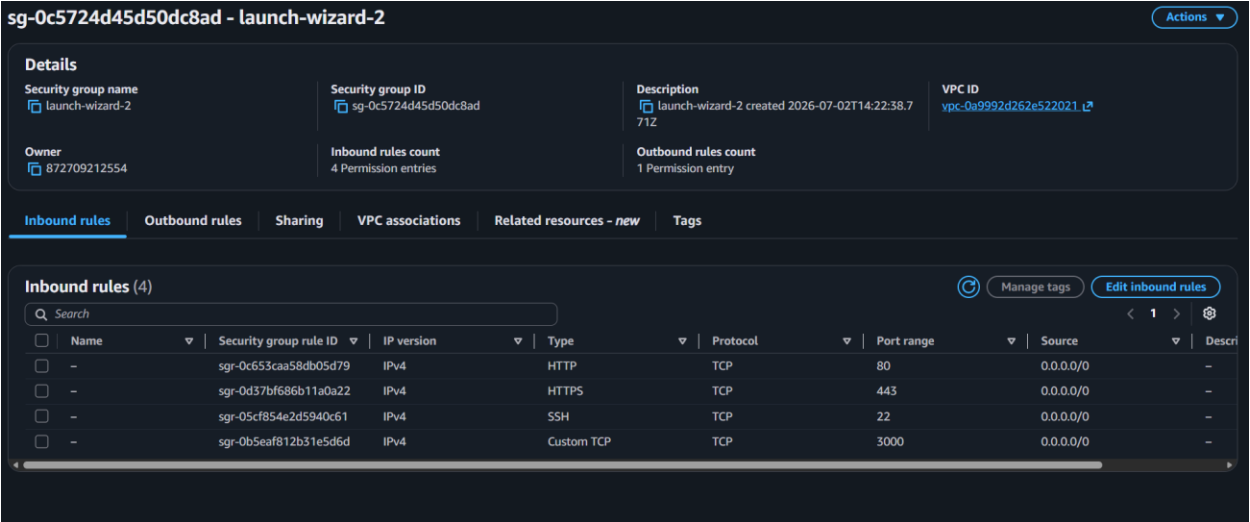


Figure: Security Group

GitHub Repository

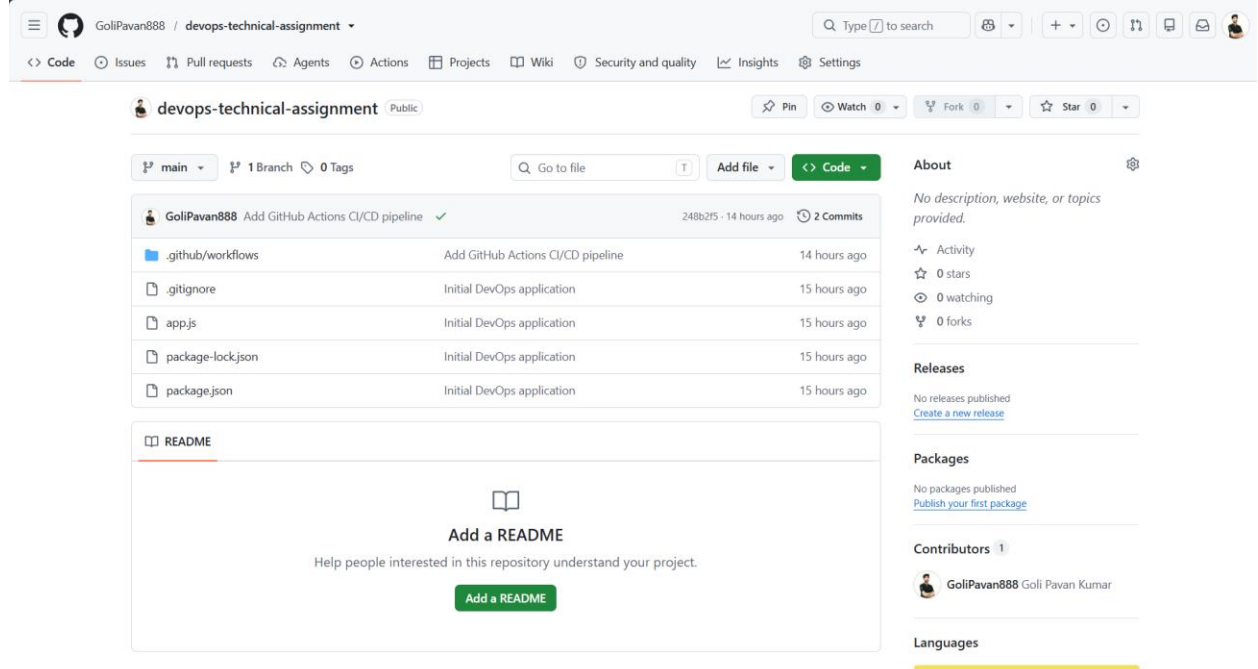


Figure: GitHub Repository

GitHub Actions CI/CD

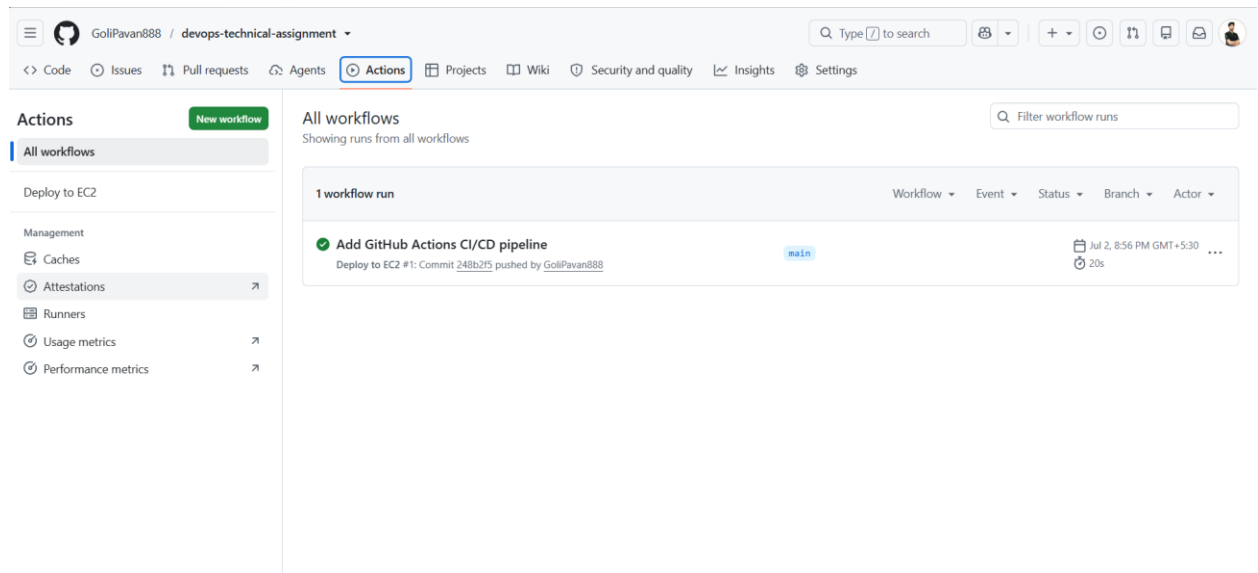


Figure: GitHub Actions CI/CD

S3 Backup Bucket

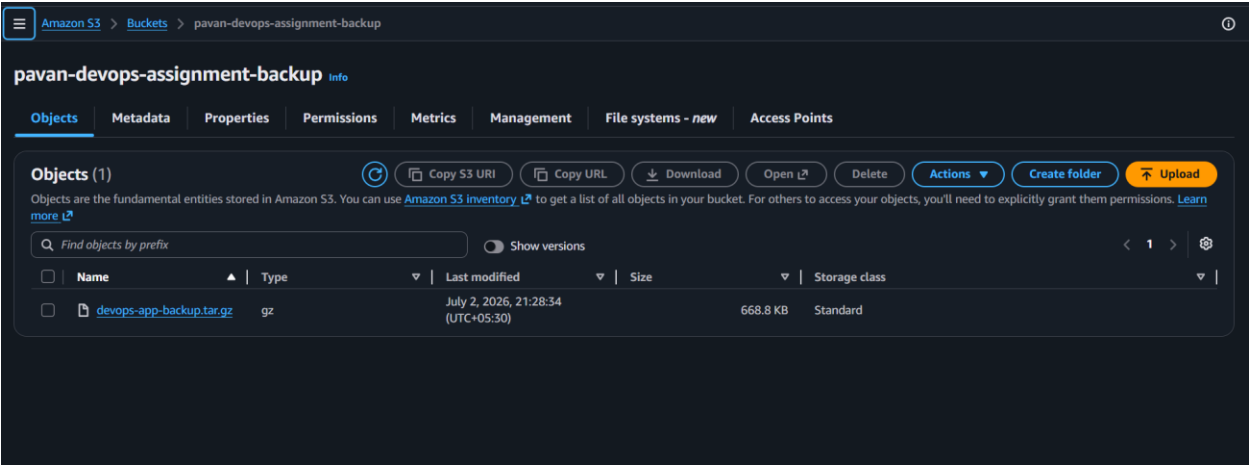


Figure: S3 Backup Bucket

CloudWatch Dashboard

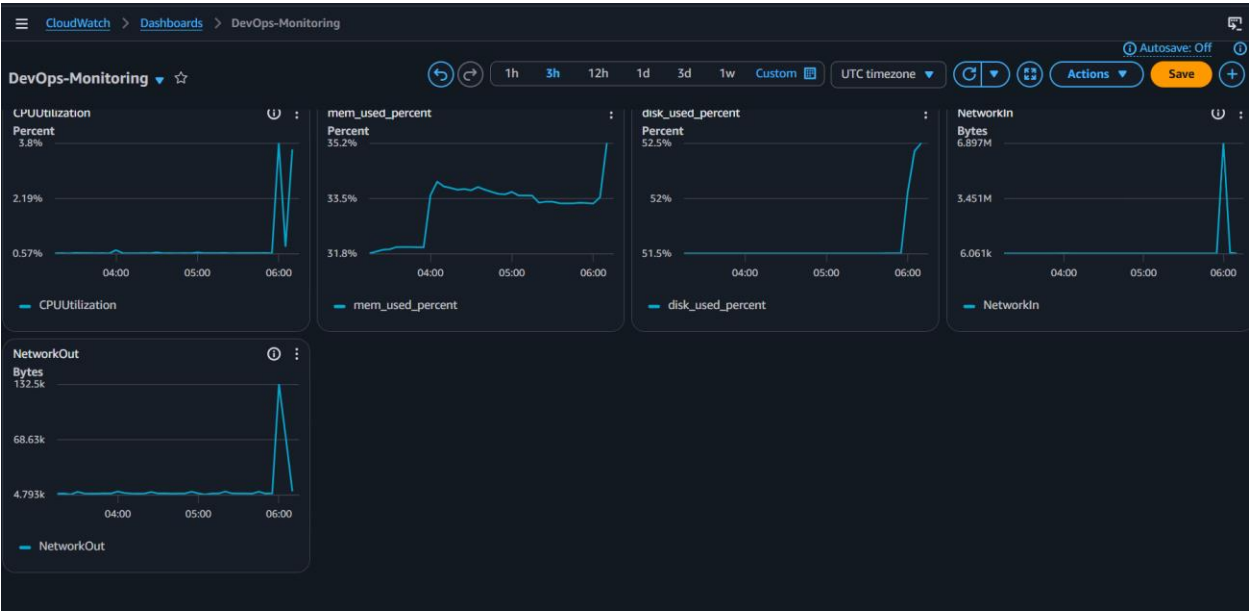


Figure: CloudWatch Dashboard

CloudWatch Alarm

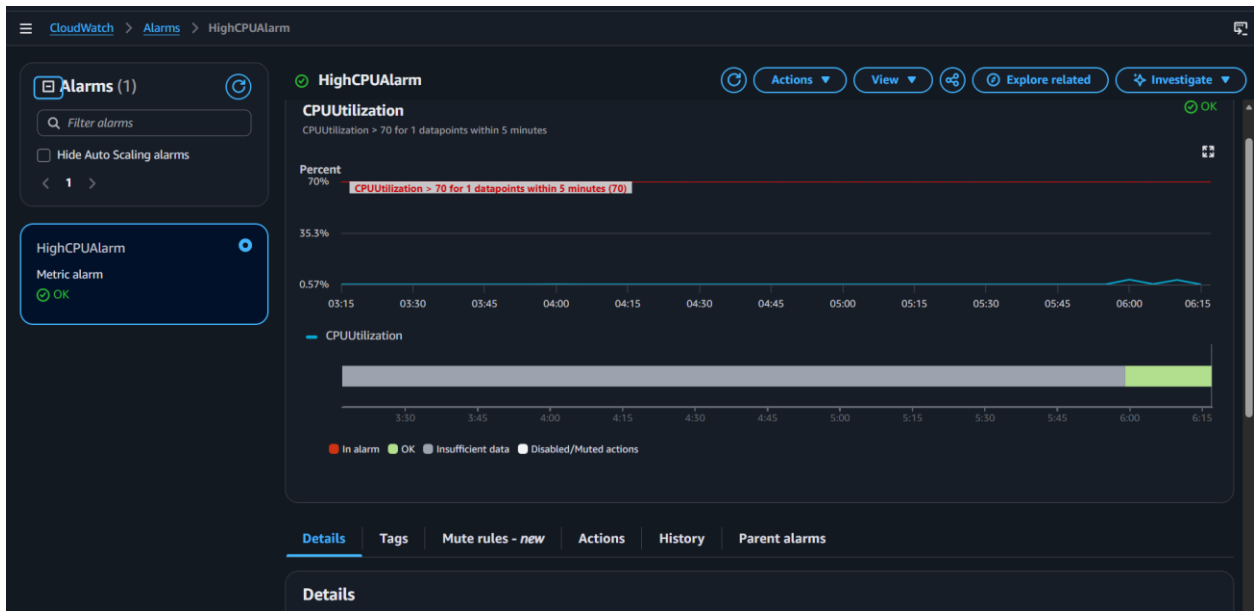


Figure: CloudWatch Alarm

k6 Load Test

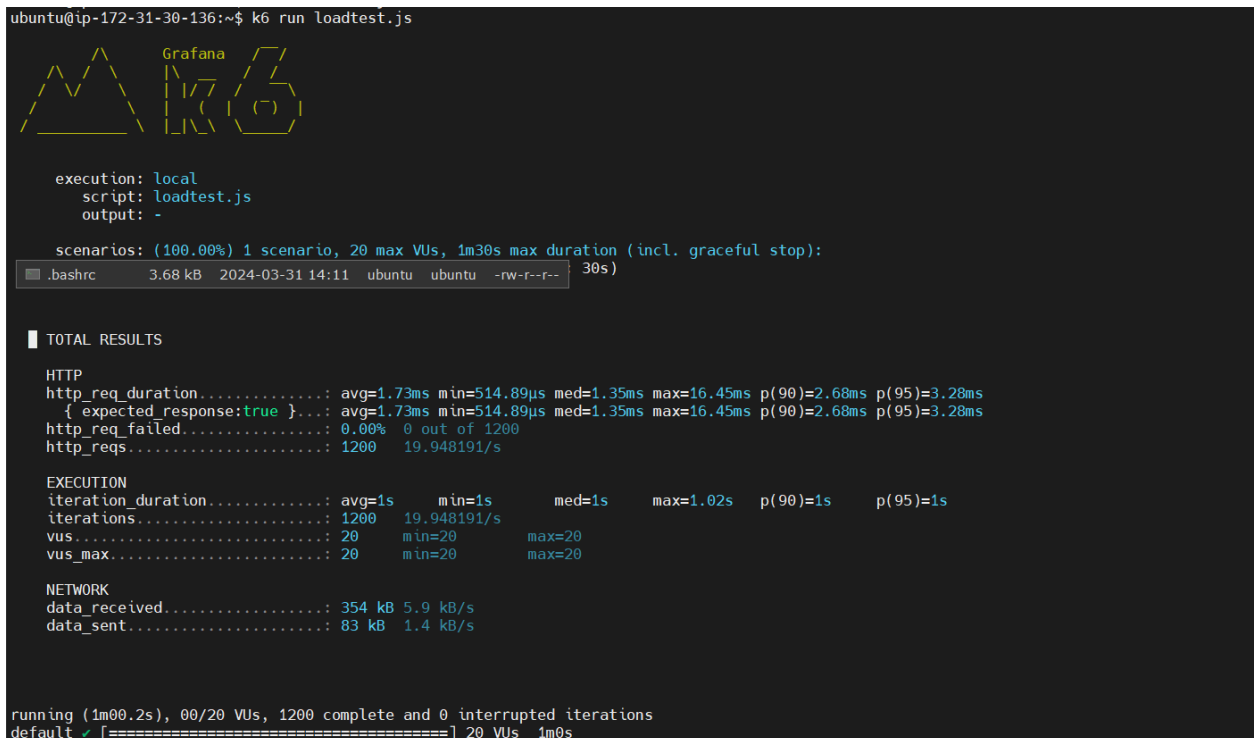


Figure: k6 Load Test

Application Running



🚀 DevOps Assignment Application is Running Successfully!

Figure: Application Running

Conclusion

This project successfully demonstrates the deployment of a production-like Node.js web application on AWS Free Tier using modern DevOps practices. The implementation includes infrastructure provisioning, automated deployment through GitHub Actions, secure access management using IAM, centralized monitoring through Amazon CloudWatch, application backup using Amazon S3, and performance validation using k6 load testing.

The project achieved all major objectives specified in the assignment while following AWS best practices for security, automation, monitoring, and operational reliability. The application successfully handled concurrent user traffic with zero failures and excellent response times.

Future enhancements include implementing HTTPS using AWS Certificate Manager, Application Load Balancer, Auto Scaling Groups, Docker containerization, Infrastructure as Code using Terraform, and deployment to Amazon ECS or Kubernetes for improved scalability and reliability.

